

Pratham Bhoot

Capstone Assignment-1: Report

Creation of Storage Account:

[Home](#) > [Storage accounts](#) >

Create a storage account ...

manage your storage account together with other resources.

Subscription *

Resource group * [Create new](#)

Instance details

Storage account name * ⓘ

Region * ⓘ [Deploy to an Azure Extended Zone](#)

Primary service ⓘ

Performance * ⓘ ☒ Standard: Recommended for most scenarios (general-purpose v2 account)
☐ Premium: Recommended for scenarios that require low latency.

Redundancy * ⓘ

Microsoft Azure Search resources, services, and docs (G+/I) Copilot tredence13@michaelklo... DEFAULT DIRECTORY (MICHAELKLO...)

Home >

pratrav Container

Search x <

+ Add Directory Upload Refresh Delete Copy Paste Rename Acquire lease Break lease Edit columns

Overview

Diagnose and solve problems

Access Control (IAM)

> Settings

pratrav > broze

Authentication method: Access key (Switch to Microsoft Entra user account)

Search blobs by prefix (case-sensitive) Only show active objects

Showing all 1 items

☐	Name	Last modified	Access tier	Blob type	Size	Lease state
☐	[.]					...
☐	retail_jot_data.csv	8/25/2025, 10:19:28 AM	Hot (Inferred)	Block blob	130.76 KiB	Available

Add or remove favorites by pressing Ctrl+Shift+F

Creation of Access Connector:

Home > Access Connector for Azure Databricks >

pratham_connector Access Connector for Azure Databricks

Search Refresh Delete

Overview

- Activity log
- Access control (IAM)
- Tags
- Resource visualizer
- Settings
 - Properties
 - Locks
 - Identity
- Automation
- Help

Essentials

Resource group (move)
[LabsKraft2027](#)

Location
West Central US

Subscription (move)
[LabsKraft](#)

Subscription ID
d008c7cc-9795-4292-bf79-74ae52cda63d

Tags (edit)
[Add tags](#)

State
Succeeded

Resource ID
/subscriptions/d008c7cc-9795-4292-bf79-74ae52cda63d/resourceGroups/...

[JSON View](#)

Adding Role Assignment:

Microsoft Azure Search resources, services, and docs (G+/)

Home > prathamstorage1 | Access Control (IAM) >

Add role assignment

Role **Members** Conditions Review + assign

Showing a filtered list of roles because your permissions include a condition. [Learn more](#)
[View my access](#)

Selected role Contributor

Assign access to
☐ User, group, or service principal
☒ Managed identity

Members + Select members

Name	Object ID
No members selected	

Description Optional

[Review + assign](#) [Previous](#) [Next](#)

Select managed identities

Some results might be hidden due to your ABAC condition.

Subscription *
[LabsKraft](#)

Managed identity
[Access Connector for Azure Databricks \(14\)](#)

[pratham](#)

Selected members:
 [pratham_connector](#)
/subscriptions/d008c7cc-9795-4292-bf79-74ae52cda63d/resourceGroups/... [Remove](#)

[Select](#) [Close](#) [Feedback](#)

Creation Of External Location:

External Locations >

Create a new external location

An external location allows you to access your data stored in cloud storage (e.g. Azure Data Lake Storage). You will need the cloud storage path and a paired credential (e.g. managed identity) which gives access to that path [Learn more](#)

External location name*

Storage type*

Azure Data Lake Storage

URL* ⓘ

Enter the bucket path that you want to use as the external location. Note: This must be an ADLS Gen2 storage account with a hierarchical namespace

 Copy from DBFS

Storage credential* [Learn more](#)

Provide a storage credential capable of accessing the URL

+ Create new storage credential

Access connector ID [Learn more](#)

User assigned managed identity ID (optional)

Creation Of New Unity Catalog:

Create a new catalog

A catalog is the first layer of Unity Catalog's three-level namespace and is used to organize your data assets. [Learn more](#)

Catalog name*

Type*

Standard

Storage location

pratham_external_location sub/path

[Create a new external location](#)

abfss://pratra@prathamstorage1.dfs.core.windows.net/

Location in cloud storage where data for managed tables will be stored. If not specified, the location will default to the metastore root location.

Cancel Create

Creation of namespace & Eventhub:

[Home](#) > [Event Hubs](#) >

 **Create Namespace** ...

Event Hubs

✓ Validation succeeded.

Basics

Namespace name	prathamevent
Subscription	LabsKraft
Resource group	LabsKraft2027
Region	West Central US
Pricing tier	Basic
Throughput Units	1
Availability Zones (Zone Redundancy)	Not Supported

Networking

Connectivity method	Public access
---------------------	---------------

Security

Minimum TLS version	1.2
Local Authentication	Enabled

 **Microsoft.Azure.SynapseAnalytics-20250825104505 | Overview** ...

Deployment

Search

Delete Cancel Redeploy Download Refresh

Overview

Inputs

Outputs

Template

✓ Your deployment is complete

Deployment name : Microsoft.Azure.SynapseAnaly... Start time : 8/25/2025, 10:47:44 AM

Subscription : LabsKraft Correlation ID : 6ac3c87d-e8c9-4f61-9a77-209...

Resource group : LabsKraft2027

> Deployment details

> Next steps

Go to resource group

Give feedback

Tell us about your experience with deployment



Cost management

Get notified to stay within your budget and prevent unexpected charges on your bill.

[Set up cost alerts >](#)



Microsoft Defender for Cloud

Secure your apps and infrastructure

[Go to Microsoft Defender for Cloud >](#)

Free Microsoft tutorials

[Start learning today >](#)

Work with an expert

Azure experts are service provider partners who can help manage your assets on Azure and be your first line of support.

Creation of Synapse:

Home > LabsKraft2027 >

pratham

Synapse workspace

Search

◊ ◀

+ New dedicated SQL pool

+ New Apache Spark pool

🔄 Refresh

🔑 Reset SQL admin password

🗑️ Delete

Overview

Activity log

Access control (IAM)

Tags

Diagnose and solve problems

Resource visualizer

Settings

Analytics pools

Security

Monitoring

Automation

Help

Suggest a workload for this Synapse workspace

×

Essentials

JSON View

Resource group (move) : LabsKraft2027

Status : Succeeded

Location : UK West

Subscription (move) : LabsKraft

Subscription ID : d008c7cc-9795-4292-bf79-74ae52cda63d

Managed virtual network : No

Managed identity object ... : d6e16efa-3aac-48f6-a36c-9734368d70b2

Workspace web URL : https://web.azuresynapse.net/workspace=%2fsubscriptions...

Tags (edit) : Add tags

Networking : Show firewall settings

Primary ADLS Gen2 acco... : https://prathamstorage2.dfs.core.windows.net

Primary ADLS Gen2 file s... : raw

SQL admin username : sqladminuser

SQL Microsoft Entra admin : tredence13@michaelkloudkraft@hotmail.onmicrosoft.com

Dedicated SQL endpoint : prathamsql.azuresynapse.net

Serverless SQL endpoint : prathamsql-on-demand.azuresynapse.net

Development endpoint : https://prathamsql.dev.azuresynapse.net

Getting started

Open Synapse Studio

Start building your fully-integrated analytics solution and unlock new insights.

Open

Read documentation

Learn how to be productive quickly. Explore concepts, tutorials, and samples.

Learn more

Analytics pools

Search to filter items...

Name

Type

Size

New SQL Pool:

New dedicated SQL pool ...

* Basics * Additional settings Tags **Review + create**

Product details

Azure Synapse Analytics dedicated SQL pool by Microsoft

Terms of use | Privacy policy

Est. Cost Per Hour

1381.03 INR

View pricing details

Terms

By clicking Create, I (a) agree to the legal terms and privacy statement(s) associated with the Marketplace offering(s) listed above; (b) authorize Microsoft to bill my current payment me same billing frequency as my Azure subscription; and (c) agree that Microsoft may share my contact, usage and transactional information with the provider(s) of the offering(s) for supp not provide rights for third-party offerings. For additional details see [Azure Marketplace Terms](#).

Basics

Dedicated SQL pool name

prathampool

Performance level

DW1000c

Additional settings

ADLS:

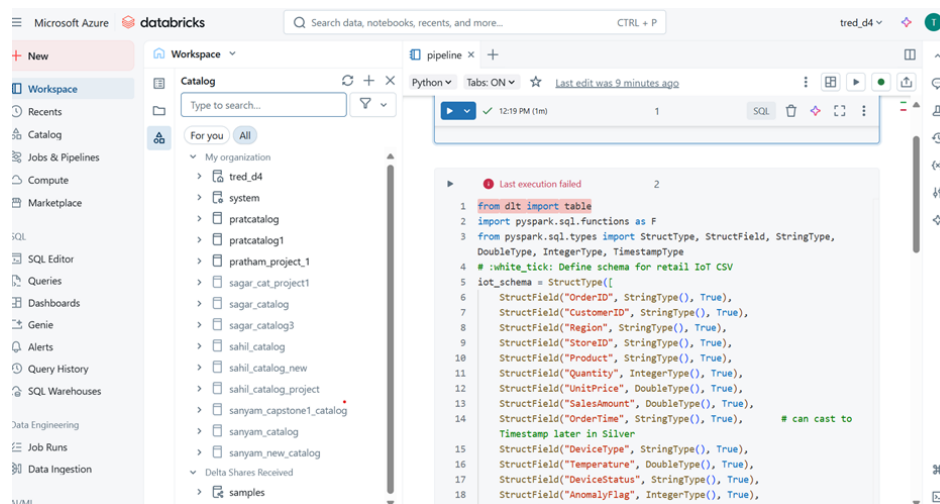
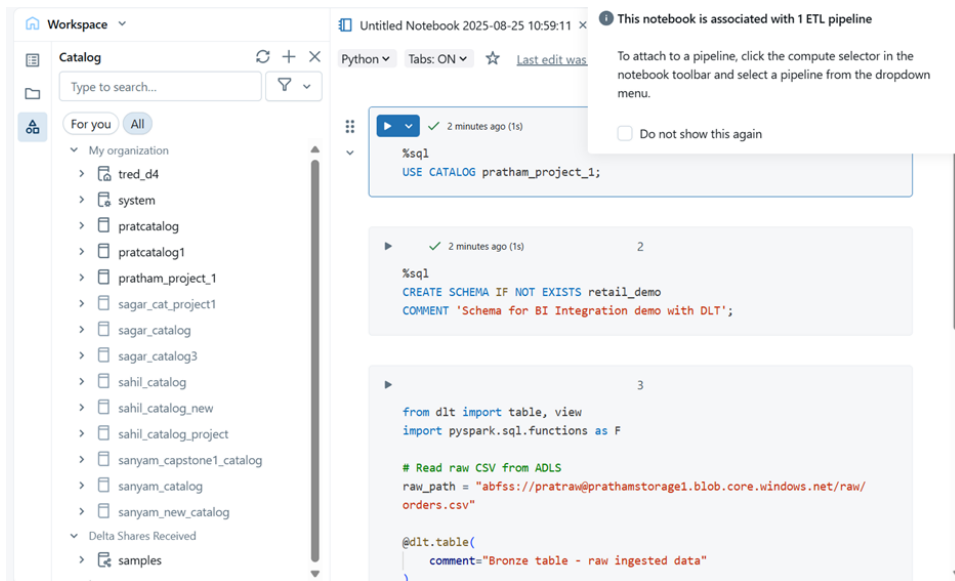
Bronze Layer:

```
from dlt import table
import pyspark.sql.functions as F
from pyspark.sql.types import StructType, StructField, StringType,
DoubleType, IntegerType, TimestampType
# :white_tick: Define schema for retail IoT CSV
iot_schema = StructType([
    StructField("OrderID", StringType(), True),
    StructField("CustomerID", StringType(), True),
    StructField("Region", StringType(), True),
    StructField("StoreID", StringType(), True),
    StructField("Product", StringType(), True),
    StructField("Quantity", IntegerType(), True),
    StructField("UnitPrice", DoubleType(), True),
    StructField("SalesAmount", DoubleType(), True),
    StructField("OrderTime", StringType(), True), # can cast to
Timestamp later in Silver
    StructField("DeviceType", StringType(), True),
    StructField("Temperature", DoubleType(), True),
    StructField("DeviceStatus", StringType(), True),
    StructField("AnomalyFlag", IntegerType(), True),
    StructField("AnomalyType", StringType(), True)
])
# :white_tick: Paths (use subdirectories to avoid overlap with DLT
system files)
raw_path = "abfss://pratr@prathamstorage1.dfs.core.windows.net/
raw_data/"
schema_path = "abfss://
pratr@prathamstorage1.dfs.core.windows.net/dlt_schemas/
bronze_orders_pratham"
checkpoint_path = "abfss://
pratr@prathamstorage1.dfs.core.windows.net/dlt_checkpoints/
bronze_orders_pratham"
@table(
    name="pratham_project_1.capstone_retail_pratham.bronze_orders",
    comment="Bronze table - raw ingested IoT retail data via Auto
Loader"
```

```

)
def bronze_orders():
    return (
        spark.readStream
            .format("cloudFiles")
            .option("cloudFiles.format", "csv")
            .option("header", True)
            .option("cloudFiles.schemaLocation", schema_path)
            .schema(iot_schema) # :white_tick: manually set schema (no
infer needed)
            .load(raw_path)    # :white_tick: points to /raw_data/ folder
    )

```



Combine of All three Layers:

```
from dlt import table
import pyspark.sql.functions as F
from pyspark.sql.types import StructType, StructField, StringType, DoubleType, IntegerType,
TimestampType
# :white_tick: Define schema for retail IoT CSV
iot_schema = StructType([
    StructField("OrderID", StringType(), True),
    StructField("CustomerID", StringType(), True),
    StructField("Region", StringType(), True),
    StructField("StoreID", StringType(), True),
    StructField("Product", StringType(), True),
    StructField("Quantity", IntegerType(), True),
    StructField("UnitPrice", DoubleType(), True),
    StructField("SalesAmount", DoubleType(), True),
    StructField("OrderTime", StringType(), True),      # can cast to Timestamp later in Silver
    StructField("DeviceType", StringType(), True),
    StructField("Temperature", DoubleType(), True),
    StructField("DeviceStatus", StringType(), True),
    StructField("AnomalyFlag", IntegerType(), True),
    StructField("AnomalyType", StringType(), True)
])
# :white_tick: Paths (use subdirectories to avoid overlap with DLT system files)
raw_path = "abfss://raw@prathamstorage2.dfs.core.windows.net/raw_data/"
schema_path = "abfss://raw@prathamstorage2.dfs.core.windows.net/dlt_schemas/
bronze_orders_pratham"
checkpoint_path = "abfss://raw@prathamstorage2.dfs.core.windows.net/dlt_checkpoints/
bronze_orders_pratham"
@table(
    name="pratham_02_project.capstone_retail_pratham.bronze_orders",
    comment="Bronze table - raw ingested IoT retail data via Auto Loader"
)
def bronze_orders():
    return (
        spark.readStream
            .format("cloudFiles")
            .option("cloudFiles.format", "csv")
            .option("header", True)
            .option("cloudFiles.schemaLocation", schema_path)
            .schema(iot_schema) # :white_tick: manually set schema (no infer needed)
            .load(raw_path)     # :white_tick: points to /raw_data/ folder
    )
# =====
# Silver Layer - Cleaned IoT Orders
# =====
@table(
    name="capstone_retail_pratham.silver_orders",
    comment="Silver layer: cleaned and deduplicated IoT orders with anomaly handling"
)
def silver_orders():
    # Read Bronze
    orders = dlt.read("pratham_02_project.capstone_retail_pratham.bronze_orders")
```



```

# Standardize schema
orders = (
    orders.withColumn("Quantity", F.col("Quantity").cast(IntegerType()))
        .withColumn("UnitPrice", F.col("UnitPrice").cast(DoubleType()))
        .withColumn("SalesAmount", F.col("SalesAmount").cast(DoubleType()))
        .withColumn("Temperature", F.col("Temperature").cast(DoubleType()))
        .withColumn("OrderTime", F.to_timestamp("OrderTime", "M/d/yyyy H:mm"))
)
# Deduplicate by OrderID
orders = orders.dropDuplicates(["OrderID"])
# Handle missing anomaly type
orders = orders.withColumn(
    "AnomalyType",
    F.when((F.col("AnomalyFlag") == 1) & (F.col("AnomalyType").isNull()), "Unknown")
        .when((F.col("AnomalyFlag") == 1) & (F.col("AnomalyType") == ""), "Unknown")
        .otherwise(F.col("AnomalyType"))
)
# Add business rule fields
orders = (
    orders.withColumn("IsAnomaly", F.col("AnomalyFlag") == 1)
        .withColumn("DeviceStatus", F.upper(F.col("DeviceStatus")))
        .withColumn("NormalizedStoreID", F.upper(F.col("StoreID")))
        .withColumn("NormalizedProduct", F.upper(F.col("Product")))
)
return orders

# =====
# Gold Layer Tables (capstone_retail.gold)
# =====
# 1. Daily / Monthly Sales per Region
@table(
    name="pratham_02_project.capstone_retail_pratham.gold_sales_region",
    comment="Daily & Monthly Sales per Region"
)
def gold_sales_region():
    return (
        dlt.read("pratham_02_project.capstone_retail_pratham.silver_orders")
        .withColumn("OrderDate", F.to_date("OrderTime", "M/d/yyyy H:mm"))
        .withColumn("YearMonth", F.date_format("OrderDate", "yyyy-MM"))
        .groupBy("Region", "OrderDate", "YearMonth")
        .agg(F.sum("SalesAmount").alias("TotalSales"))
    )
# 2. Device Anomaly Trend per Store
@table(
    name="pratham_02_project.capstone_retail_pratham.gold_anomaly_trends",
    comment="Device Anomaly Trend per Store"
)
def gold_anomaly_trends():
    return (
        dlt.read("pratham_02_project.capstone_retail_pratham.silver_orders")
        .withColumn("OrderDate", F.to_date("OrderTime", "M/d/yyyy H:mm"))
        .groupBy("StoreID", "OrderDate", "DeviceType", "AnomalyType")
        .agg(F.count("*").alias("AnomalyCount"))
    )

```

3. Conversion Rate Impact when devices fail

```
@table(
    name="pratham_02_project.capstone_retail_pratham.gold_conversion_impact",
    comment="Conversion rate impact of device failures"
)
def gold_conversion_impact():
    df = dlt.read("pratham_02_project.capstone_retail_pratham.silver_orders")
    total_orders = df.groupBy("StoreID").agg(F.count("*").alias("TotalOrders"))
    failed_orders = (
        df.filter(df.AnomalyFlag == 1)
        .groupBy("StoreID")
        .agg(F.count("*").alias("FailedOrders"))
    )
    return (
        total_orders.join(failed_orders, "StoreID", "left")
        .withColumn("FailedOrders", F.coalesce(F.col("FailedOrders"), F.lit(0)))
        .withColumn("FailureRate", F.col("FailedOrders") / F.col("TotalOrders"))
    )
```

4. Store Tier Classification

```
@table(
    name="pratham_02_project.capstone_retail_pratham.gold_store_tiers",
    comment="Store tiers based on total sales (High / Medium / Low performers)"
)
def gold_store_tiers():
    df = (
        dlt.read("pratham_02_project.capstone_retail_pratham.silver_orders")
        .groupBy("StoreID")
        .agg(F.sum("SalesAmount").alias("TotalSales"))
    )
    return (
        df.withColumn(
            "StoreTier",
            F.when(F.col("TotalSales") > 100000, "High")
            .when(F.col("TotalSales") > 50000, "Medium")
            .otherwise("Low")
        )
    )
```

5. Weighted Average Sales vs Anomalies

```
@table(
    name="pratham_02_project.capstone_retail_pratham.gold_weighted_sales_anomalies",
    comment="Weighted average of sales vs anomalies per store"
)
def gold_weighted_sales_anomalies():
    df = dlt.read("pratham_02_project.capstone_retail_pratham.silver_orders")
    return (
        df.groupBy("StoreID")
        .agg(
            F.sum("SalesAmount").alias("TotalSales"),
            F.sum(F.when(F.col("AnomalyFlag") == 1, 1).otherwise(0)).alias("TotalAnomalies")
        )
        .withColumn("SalesPerAnomaly",
            F.when(F.col("TotalAnomalies") > 0, F.col("TotalSales") / F.col("TotalAnomalies"))
            .otherwise(F.lit(None))
        )
    )
```

Jobs & Pipelines >

pratham ☆ Send feedback

Run: 8/25/2025, 7:43:47 PM · Completed Select tables for refresh

Graph List

Pipeline details Update details

Pipeline ID: 4e224df8-2b4b-43ef-93e1-88ea84e3b50b

Pipeline type: ETL pipeline

Source code: /Users/tredence13@michaelkloudkrafthotmail.onmicrosoft.com/Untitled Notebook 2025-08-25 15:19:44

Run as: Tredence LabsKraft13

Fit to viewport: None

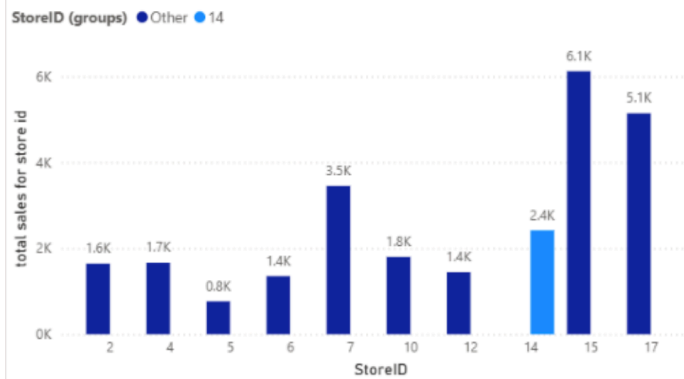
Event log Query history

All Info Warning Error Filter...

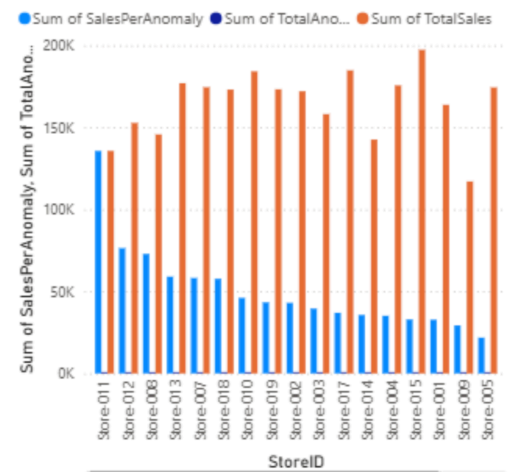
- 2 minutes ago user_action User tredence13@michaelkloudkrafthotmail.onmicrosoft.com started an update.
- 2 minutes ago create_update Update 1ed075 started by USER_ACTION.
- 2 minutes ago update_progress Update 1ed075 is INITIALIZING.

Power BI Outputs:

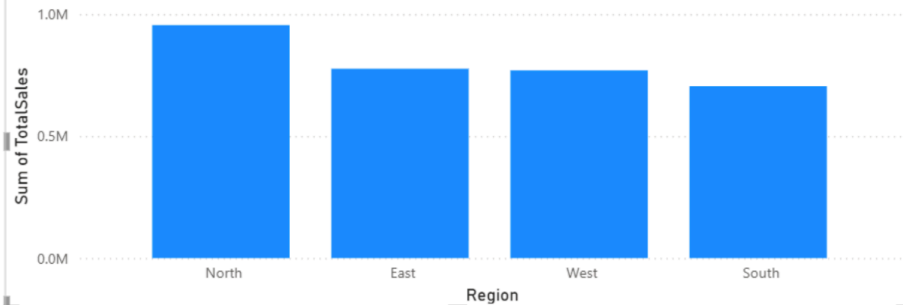
total sales for store id by StoreID and StoreID (groups)



Sum of SalesPerAnomaly, Sum of TotalAnomalies and Sum of TotalSales by StoreID



Sum of TotalSales by Region



Creation of Event Namespace & EventHub:

Home > Event Hubs >

Create Namespace

✓ Validation succeeded.

Basics

Namespace name	prathamevent
Subscription	LabsKraft
Resource group	LabsKraft2027
Region	West Central US
Pricing tier	Basic
Throughput Units	1
Availability Zones (Zone Redundancy)	Not Supported

Networking

Connectivity method	Public access
---------------------	---------------

Security

Minimum TLS version	1.2
Local Authentication	Enabled

Creation Of Virtual Machine:

Home >

prathamvm

Virtual machine

Help me copy this VM in any region

Manage this VM with Azure CLI

×

Search

Connect

Start

Restart

Stop

Hibernate

Capture

Delete

Refresh

Open in mobile

Feedback

CLI / PS

Overview

Activity log

Access control (IAM)

Tags

Diagnose and solve problems

Resource visualizer

Connect

Networking

Settings

Availability + scale

Security

Backup + disaster recovery

Operations

Monitoring

Automation

Help

Essentials

Resource group (move) : LabsKraft2027

Status : Running

Location : UK West

Subscription (move) : LabsKraft

Subscription ID : d008c7cc-9795-4292-bf79-74ae52cda63d

Operating system : Linux (ubuntu 24.04)

Size : Standard B1s (1 vcpu, 1 GiB memory)

Public IP address : 20.162.116.225

Virtual network/subnet : prathamvm-vnet/default

DNS name : Not configured

Health state : -

Time created : 25/08/2025, 14:36 UTC

Tags (edit) : Add tags

JSON View

Properties

Monitoring

Capabilities (7)

Recommendations

Tutorials

Virtual machine

Computer name : prathamvm

Operating system : Linux (ubuntu 24.04)

VM generation : V2

VM architecture : x64

Agent status : Ready

Agent version : 2.14.0.1

Hibernation : Disabled

Host group : -

Host : -

Networking

Public IP address : 20.162.116.225 (Network interface prathamvm992)

Public IP address (IPv6) : -

Private IP address : 10.0.0.4

Private IP address (IPv6) : -

Virtual network/subnet : prathamvm-vnet/default

DNS name : Configure

Size

Size : Standard B1s

Add or remove favorites by pressing **Cmd+Shift+F**

Code of GNU nano:

```
GNU nano 7.2                                iot_stream.py
import random
import time
import uuid
from datetime import datetime
from azure.eventhub import EventHubProducerClient,EventData

# CONFIGURATION
EVENT_HUB_CONNECTION_STR = "Endpoint=sb://prathamevent.servicebus.windows.net/;SharedAccessKeyName=pratham11;SharedAccessKey=rQdXpCnN03bKWtiRPYpAMoIPxi8lUrTEH+AEhJr34+cub"
EVENT_HUB_NAME = "prathameventb"

# Function to generate one IoT row
def generate_row():
    quantity = random.randint(1, 10)
    unit_price = round(random.uniform(50, 600), 2)
    sales_amount = round(quantity * unit_price, 2)
    device_status = random.choice(["Active", "Inactive", "Error"])
    anomaly_flag = 1 if device_status != "Active" else 0
    anomaly_type = random.choice(["Overheat", "LowBattery", "Disconnected"]) if anomaly_flag else "None"
    return {
        "OrderID": str(uuid.uuid4()),
        "CustomerID": f"CUST{random.randint(1000, 9999)}",
        "Region": random.choice(["North", "South", "East", "West"]),
        "StoreID": random.choice(["Store001", "Store002", "Store003"]),
        "Product": random.choice(["Laptop", "Phone", "Monitor", "Keyboard", "Mouse"]),
        "Quantity": quantity,
        "UnitPrice": unit_price,
        "SalesAmount": sales_amount,
        "OrderTime": datetime.now().strftime("%Y-%m-%d %H:%M:%S"),
        "DeviceType": random.choice(["SensorA", "SensorB", "SensorC"]),
        "Temperature": round(random.uniform(20, 100), 2),
        "DeviceStatus": device_status,
        "AnomalyFlag": anomaly_flag,
        "AnomalyType": anomaly_type
    }

# Event Hub Producer
producer = EventHubProducerClient.from_connection_string(
    conn_str=EVENT_HUB_CONNECTION_STR,
    eventhub_name=EVENT_HUB_NAME
)

# Send data every second
try:
    for _ in range(1000): # Send exactly 1000 rows
        row = generate_row()
        event_data = EventData(str(row)) # convert dict to string
        with producer:
            producer.send_batch([event_data])
        print(f"Sent row: {row}")
        time.sleep(1) # 1-second interval
except KeyboardInterrupt:
    print("Streaming stopped by user.")
finally:
    producer.close()
```

Home >

prathamevent Summarize properties of this Event Hubs Namespace. Show me metrics for this Event Hubs Namespace. How do I troubleshoot issues with this Event Hubs Namespace?

Event Hubs Namespace

Search Event Hub Delete Refresh Give feedback

Overview

You can start generating test data or inspect data that has already been sent with the new Azure Event Hubs Data Explorer. Click on this message to try the feature!

Activity log

Access control (IAM)

Tags

Diagnose and solve problems

Data Explorer

Resource visualizer

Events

> Settings

> Entities

> Monitoring

> Automation

> Help

Essentials JSON View

Resource group (move) : [LabsKraft2027](#)

Status : Succeeded

Location : UK West

Subscription (move) : [LabsKraft](#)

Subscription ID : d008c7cc-9795-4292-bf79-74ae52cda63d

Host name : prathamevent.servicebus.windows.net

Tags (edit) : [Add tags](#)

Created : Monday 25 August 2025 at 20:13:07 GMT+5:30

Updated : Monday 25 August 2025 at 20:13:35 GMT+5:30

Zone Redundancy : Not Enabled

Pricing tier : [Basic](#)

Throughput Units : [1 unit](#)

Auto-inflate throughput ... : [Not Supported](#)

Local Authentication : [Enabled](#)

NAMESPACE CONTENTS **KAFKA SURFACE**

0 EVENT HUBS **NOT SUPPORTED**

Show data for the last: 1 hour 6 hours 12 hours 1 day 7 days 30 days

Requests

1

0.8

0.6

0.4

0.2

Messages

100

90

80

70

60

50

40

30

20

Throughput

100B

90B

80B

70B

60B

50B

40B

30B

20B

Add or remove favorites by pressing Cmd+Shift+F

Code of EventHub Config:

nano iot_stream.py

```
import random
import time
import uuid
from datetime import datetime
from azure.eventhub import EventHubProducerClient, EventData

# -----
# CONFIGURATION
# -----
Event connection string
EVENT_HUB_NAME = "prathamhub"

# -----
# Function to generate one IoT row
# -----
def generate_row():
    quantity = random.randint(1, 10)
    unit_price = round(random.uniform(50, 500), 2)
    sales_amount = round(quantity * unit_price, 2)
    device_status = random.choice(["Active", "Inactive", "Error"])
    anomaly_flag = 1 if device_status != "Active" else 0
    anomaly_type = random.choice(["Overheat", "LowBattery", "Disconnected"]) if anomaly_flag
    else "None"

    return {
        "OrderID": str(uuid.uuid4()),
        "CustomerID": f"CUST{random.randint(1000,9999)}",
        "Region": random.choice(["North", "South", "East", "West"]),
        "StoreID": random.choice(["Store001", "Store002", "Store003"]),
        "Product": random.choice(["Laptop", "Phone", "Monitor", "Keyboard", "Mouse"]),
        "Quantity": quantity,
        "UnitPrice": unit_price,
        "SalesAmount": sales_amount,
        "OrderTime": datetime.now().strftime("%Y-%m-%d %H:%M:%S"),
        "DeviceType": random.choice(["SensorA", "SensorB", "SensorC"]),
        "Temperature": round(random.uniform(20, 100), 2),
        "DeviceStatus": device_status,
        "AnomalyFlag": anomaly_flag,
        "AnomalyType": anomaly_type
    }

# -----
# Event Hub Producer
# -----
producer = EventHubProducerClient.from_connection_string(
    conn_str=EVENT_HUB_CONNECTION_STR,
    eventhub_name=EVENT_HUB_NAME
```

```

)

# -----
# Send data every second
# -----
try:
    for _ in range(1000): # Send exactly 1000 rows
        row = generate_row()
        event_data = EventData(str(row)) # convert dict to string
        with producer:
            producer.send_batch([event_data])
        print(f"Sent row: {row}")
        time.sleep(1) # 1-second interval
except KeyboardInterrupt:
    print("Streaming stopped by user.")
finally:
    producer.close()

```

Code :

```

# =====
# Bronze → Silver → Gold with Event Hub Integration
# =====
from dlt import table, read
import pyspark.sql.functions as F
from pyspark.sql.types import *
# =====
# :one: Bronze Layer: Schema & CSV
# =====
iot_schema = StructType([
    StructField("OrderID", StringType(), True),
    StructField("CustomerID", StringType(), True),
    StructField("Region", StringType(), True),
    StructField("StoreID", StringType(), True),
    StructField("Product", StringType(), True),
    StructField("Quantity", IntegerType(), True),
    StructField("UnitPrice", DoubleType(), True),
    StructField("SalesAmount", DoubleType(), True),
    StructField("OrderTime", StringType(), True),
    StructField("DeviceType", StringType(), True),
    StructField("Temperature", DoubleType(), True),
    StructField("DeviceStatus", StringType(), True),
    StructField("AnomalyFlag", IntegerType(), True),
    StructField("AnomalyType", StringType(), True)
])
raw_path = "abfss://raw@prathamstorage2.dfs.core.windows.net/incoming/"
schema_path = "abfss://raw@prathamstorage2.dfs.core.windows.net/dlt_schemas/"
bronze_orders_pratham =
@table(
    name="pratham_02_project.bronze.bronze_orders_csv",

```

```

    comment="Bronze table - raw IoT CSV"
)
def bronze_orders_csv():
    return (
        spark.readStream
            .format("cloudFiles")
            .option("cloudFiles.format", "csv")
            .option("header", True)
            .option("cloudFiles.schemaLocation", schema_path)
            .schema(iot_schema)
            .load(raw_path)
            .withColumn("OrderTime", F.to_timestamp("OrderTime"))
    )
# =====
# :two: Bronze Layer: Event Hub IoT
# =====
eventhub_connection_string = "Endpoint=sb://
prathamevent.servicebus.windows.net/;SharedAccessKeyName=manage;SharedAccessKey=F
KKqGhzAyDEqYXgNZvxzpYat2gUjW1VTS+AEhAlkesY=;EntityPath=event-vm"
consumer_group = "$Default"
ehConf = {
    'eventhubs.connectionString': eventhub_connection_string,
    'eventhubs.consumerGroup': consumer_group,
    'eventhubs.startingPosition': '@latest'
}
@table(
    name="pratham_02_project.bronze.bronze_orders_eventhub",
    comment="Bronze table - live IoT data from Event Hub"
)
def bronze_orders_eventhub():
    df = spark.readStream \
        .format("eventhubs") \
        .options(**ehConf) \
        .load()
    # Event Hub body is bytes → convert to string → parse JSON
    df1 = df.selectExpr("cast(body as string) as json_string")
    df_parsed = df1.select(F.from_json(F.col("json_string"),
iot_schema).alias("data")).select("data.*")
    return df_parsed.withColumn("OrderTime", F.to_timestamp("OrderTime"))
# =====
# :three: Unified Bronze Table
# =====
@table(
    name="pratham_02_project.capstone_retail_pratham.bronze_orders",
    comment="Unified Bronze table (CSV + Event Hub)"
)
def bronze_orders():
    df_csv = read("pratham_02_project.capstone_retail_pratham.bronze_orders_csv")
    df_eh = read("pratham_02_project.capstone_retail_pratham.bronze_orders_eventhub")
    return df_csv.unionByName(df_eh)
# =====
# :four: Silver Layer
# =====

```



```

@table(
    name="pratham_02_project.capstone_retail_pratham.silver_orders",
    comment="Silver layer: cleaned and deduplicated IoT orders with anomaly handling"
)
def silver_orders():
    orders = read("pratham_02_project.capstone_retail_pratham.bronze_orders")
    orders = (
        orders.withColumn("Quantity", F.col("Quantity").cast(IntegerType()))
        .withColumn("UnitPrice", F.col("UnitPrice").cast(DoubleType()))
        .withColumn("SalesAmount", F.col("SalesAmount").cast(DoubleType()))
        .withColumn("Temperature", F.col("Temperature").cast(DoubleType()))
        .withColumn("OrderTime", F.to_timestamp("OrderTime", "M/d/yyyy H:mm"))
    )
    orders = orders.dropDuplicates(["OrderID"])
    orders = orders.withColumn(
        "AnomalyType",
        F.when((F.col("AnomalyFlag") == 1) & (F.col("AnomalyType").isNull()), "Unknown")
        .when((F.col("AnomalyFlag") == 1) & (F.col("AnomalyType") == ""), "Unknown")
        .otherwise(F.col("AnomalyType"))
    )
    orders = (
        orders.withColumn("IsAnomaly", F.col("AnomalyFlag") == 1)
        .withColumn("DeviceStatus", F.upper(F.col("DeviceStatus")))
        .withColumn("NormalizedStoreID", F.upper(F.col("StoreID")))
        .withColumn("NormalizedProduct", F.upper(F.col("Product")))
    )
    return orders
# =====
# :five: Gold Layer
# =====
@table(
    name="pratham_02_project.capstone_retail_pratham.gold_sales_region",
    comment="Daily & Monthly Sales per Region"
)
def gold_sales_region():
    return (
        read("pratham_02_project.capstone_retail_pratham.silver_orders")
        .withColumn("OrderDate", F.to_date("OrderTime", "M/d/yyyy H:mm"))
        .withColumn("YearMonth", F.date_format("OrderDate", "yyyy-MM"))
        .groupBy("Region", "OrderDate", "YearMonth")
        .agg(F.sum("SalesAmount").alias("TotalSales"))
    )
@table(
    name="pratham_02_project.capstone_retail_pratham.gold_anomaly_trends",
    comment="Device Anomaly Trend per Store"
)
def gold_anomaly_trends():
    return (
        read("pratham_02_project.capstone_retail_pratham.silver_orders")
        .withColumn("OrderDate", F.to_date("OrderTime", "M/d/yyyy H:mm"))
        .groupBy("StoreID", "OrderDate", "DeviceType", "AnomalyType")
        .agg(F.count("*").alias("AnomalyCount"))
    )
@table(

```

```

name="pratham_02_project.capstone_retail_pratham.gold_conversion_impact",
comment="Conversion rate impact of device failures"
)
def gold_conversion_impact():
    df = read("pratham_02_project.capstone_retail_pratham.silver_orders")
    total_orders = df.groupBy("StoreID").agg(F.count("*").alias("TotalOrders"))
    failed_orders = (
        df.filter(df.AnomalyFlag == 1)
        .groupBy("StoreID")
        .agg(F.count("*").alias("FailedOrders"))
    )
    return (
        total_orders.join(failed_orders, "StoreID", "left")
        .withColumn("FailedOrders", F.coalesce(F.col("FailedOrders"), F.lit(0)))
        .withColumn("FailureRate", F.col("FailedOrders") / F.col("TotalOrders"))
    )
)
@table(
    name="pratham_02_project.capstone_retail_pratham.gold_store_tiers",
    comment="Store tiers based on total sales (High / Medium / Low performers)"
)
def gold_store_tiers():
    df = (
        read("pratham_02_project.capstone_retail_pratham.silver_orders")
        .groupBy("StoreID")
        .agg(F.sum("SalesAmount").alias("TotalSales"))
    )
    return (
        df.withColumn(
            "StoreTier",
            F.when(F.col("TotalSales") > 100000, "High")
            .when(F.col("TotalSales") > 50000, "Medium")
            .otherwise("Low")
        )
    )
)
@table(
    name="pratham_02_project.capstone_retail_pratham.gold_weighted_sales_anomalies",
    comment="Weighted average of sales vs anomalies per store"
)
def gold_weighted_sales_anomalies():
    df = read("pratham_02_project.capstone_retail_pratham.silver_orders")
    return (
        df.groupBy("StoreID")
        .agg(
            F.sum("SalesAmount").alias("TotalSales"),
            F.sum(F.when(F.col("AnomalyFlag") == 1, 1).otherwise(0)).alias("TotalAnomalies")
        )
        .withColumn("SalesPerAnomaly",
            F.when(F.col("TotalAnomalies") > 0, F.col("TotalSales") / F.col("TotalAnomalies"))
            .otherwise(F.lit(None))
        )
    )
)

```

Jobs & Pipelines >

pratham1 ☆ Send feedback

Development Production Settings Schedule Share Start

Run: 8/25/2025, 9:04:53 PM · Failed Select tables for refresh

Creating update
Waiting for resources
Initializing
4 Setting up tables
5 Rendering graph

Pipeline details Update details

Pipeline ID 16e3ea5b-c85b-4291-bc83-a1feb5ee9088

Pipeline type ETL pipeline

Source code /Users/tredence13@michaelkloudkrafthotmail.onmicrosoft.com/Untitled Notebook 2025-08-25 20:36:05

Run as Tredence LabsKraft13

Tags None

Event log Query history

All Info Warning Error Filter...

2 minutes ago user_action User tredence13@michaelkloudkrafthotmail.onmicrosoft.com started an update.

2 minutes ago create_update Update 574c64 started by USER_ACTION.

2 minutes ago update_progress Update 574c64 is INITIALIZING.
Update 574c64 is INITIALIZING.

1 minute ago flow_progress Failed to resolve flow due to upstream failure: 'pratham_02_project.capstone_retail_pratham.silver_orders'.

Getting the Above Error.

Event log Query history

All Info Warning Error Filter...

1 hour ago flow_progress Failed to resolve flow due to upstream failure: 'pratham_02_project.capstone_retail_pratham.gold_sales_region'.

1 hour ago flow_progress Failed to resolve flow: 'retail_catalog_pratham.bronze.bronze_orders_eventhub'.

1 hour ago flow_progress Failed to resolve flow: 'pratham_02_project.capstone_retail_pratham.bronze_orders'.

1 hour ago update_progress Update 574c64 has failed. Failed to analyze flow 'retail_catalog_pratham.bronze.bronze_orders_eventhub' and 1 other flow(s)..